

Invisibook: Privacy-Preserving, Censorship-Resistant, Decentralized Order Book

I. INTRODUCTION

Decentralized exchanges (DEXs) have grown into one of the most heavily used classes of blockchain applications, and limit order books have become their dominant trading architecture. A DEX processes an order in the same three stages as a traditional centralized exchange (CEX). In *placement*, a trader submits a buy or sell order specifying the trading pair, the side, the limit price, and the order quantity. In *matching*, the order book pairs crossing orders under a deterministic priority rule, typically price priority followed by time priority or some protocol-specific tie-breaking policy. In *settlement*, the executed trade is cleared: the corresponding assets are transferred between the two parties, their balances are updated, and the affected orders are amended or removed from the book. What separates a DEX from a CEX is that no trusted third party acts as trading intermediary or asset custodian [1]: the order book and every transition applied to it are maintained by the ledger and re-validated by every node. This is what gives a DEX its two defining advantages over a CEX: *transparency*, since the state of the market is public and independently verifiable rather than asserted by an operator, and *censorship resistance*, since no single entity can selectively refuse, reorder, or withhold a trader's order.

However, the openness that yields these advantages also creates a privacy problem that a CEX does not have. On a public order book, that openness also publishes in plaintext the quantity of every resting order and the balance of every account, which enables a class of adversarial strategies conditioned on order size. Exposing a large order reveals both its sustained trading intent and its potential price impact, and a *parasitic trader* can act on that disclosure: it accumulates a position in the same direction, taking the liquidity the large trader would otherwise have obtained, and then supplies that liquidity back at worse prices, raising the execution cost borne by the large order [2], [3]. The attack therefore has a necessary first step, *target selection*, in which the adversary must find a large order and tell it apart from the ordinary orders around it, since positioning is costly and only repays itself above a size threshold. Public on-chain order books such as dYdX [4] and Injective [5] publish the field that makes this step trivial, and

they publish it under conditions strictly worse than a CEX: the book and its entire history are permanently readable by anyone at no cost, replayable off-line, and accompanied by account balances that expose each trader's latent capacity to trade even before an order is placed.

Existing privacy-preserving exchange designs fall into three classes, each of which surrenders something essential. Systems built around an *off-chain or centralized matcher* keep order details away from the public, but reintroduce a trusted entity into the trading workflow: the matcher learns the sensitive order information it is supposed to protect, and its discretion over what to include weakens censorship resistance [6], [7]. *Fully hiding* designs conceal balances, sides, prices, and quantities alike, and thereby operate as dark pools; but withholding pre-trade information removes price-relevant signals from the public market, and empirical evidence shows that a high level of dark trading degrades price discovery and raises adverse-selection risk in the lit market [8]. *Iceberg orders* provide an intermediate form of privacy by exposing only a limited visible quantity and replenishing it as executions occur; however, the hidden quantity is protected by the trading venue's order-handling mechanism rather than by cryptographic guarantees. Both the presence and the magnitude of hidden liquidity are predictable from observable order-book dynamics [9], [10], and detection algorithms recover an iceberg's hidden volume from the replenishment pattern its own execution leaves behind [11], [12]. It therefore remains an open problem to protect the privacy of order quantities and balances while preserving both the decentralization of the venue and the public price discovery a lit order book exists to provide.

This paper answers that question with Invisibook, and the answer begins from a deliberate separation of what an order book publishes from what it must conceal. To preserve price discovery, we publish every order's trading pair, side, limit price, and fee in plaintext, so that matching remains an ordinary deterministic priority rule executed and verified by the chain in public. To protect the trader, we publish no quantity at all: an order carries its quantity only as a hiding, binding commitment, balances are held in a shielded representation, and every quantity-dependent step is deferred to a maliciously secure two-party computation run by the two matched traders themselves, whose result the chain verifies through a collaborative zero-knowledge proof [13] anchored to the on-chain commitments.

This choice raises two technical challenges, and we outline our answer to each. *Matching without arithmetic*. The first challenge is that a chain holding quantities as commitments

can no longer perform the arithmetic that matching requires. It cannot compare, split, or sum committed quantities, and so cannot walk the book to fill an incoming order, nor even determine when that order has been exhausted. Matching must therefore be redefined so that it remains deterministic and publicly verifiable under this restriction. We make matching strictly pairwise: the chain pairs an incoming order with the single highest-priority crossing order on the public price field alone, and defers every quantity-dependent step to settlement. A partially filled order re-enters the book under a fresh commitment to its residual, so it continues to compete for priority without disclosing how much of it remains. *A counterparty that is also the adversary.* The second challenge is that, with no server and no trusted third party, the party a trader must be protected against is the same party it computes with. A single corruption out of two participants is a dishonest majority, so the protocol must hold against a party that deviates arbitrarily, and it must do so in three respects. (i) The computation’s inputs must be bound to the quantities previously committed on chain. (ii) Neither party may corrupt the result the chain publishes while privately learning the correct one. (iii) Neither party may stall: a matched pair is locked and can no longer be cancelled, so withholding a single message freezes the counterparty’s order and capital, and no cryptographic check detects silence. We address (i) with a collaborative zero-knowledge proof whose statement anchors the computation’s inputs to the on-chain commitments, (ii) with a two-step on-chain assembly that verifies the proof before any value is opened, and (iii) with an on-chain challenge-and-freeze mechanism that makes delay costly to the delaying party while releasing the compliant one.

We summarize our contributions as follows.

- *A amount-hidden DEX architecture.* To prevent target selection from exploiting publicly visible order quantities, we propose an order-book DEX that reveals the order side and limit price for price formation, while hiding the quantity under a cryptographic commitment. Matching depends only on the plaintext fields and follows a deterministic priority rule executed on-chain and re-validated by every node. By the hiding property of the commitment scheme, order quantity remains computationally hidden, so attacking strategies cannot exploit the order size, while the venue retains transparent and censorship-resistant matching.
- *A settlement protocol secure in the malicious setting.* To prevent either trader from learning the counterparty’s private information during settlement, we propose a maliciously secure two-party settlement protocol that operates directly on committed quantities. The protocol integrates an SPDZ-style two-party MPC scheme with a ZK-SNARK to privately execute and prove off-chain order comparison and settlement. Following the off-chain computation, a two-step on-chain assembly procedure verifies the proof before committing any state changes, thereby ensuring atomicity of the settlement. In addition,

an on-chain challenge-and-freeze mechanism discourages strategic aborts and protocol stalling by imposing penalties on non-responsive parties.

- *Implementation and evaluation.* We implement the two-party comparison and collaborative proof protocol, its integration with single-prover settlement circuits, and a proof-verifying blockchain node. Our evaluation measures cryptographic overhead, sensitivity to network latency, and the end-to-end and on-chain costs of a complete trade. In our experimental setup, the collaborative protocol completes in 4.54 s and produces a 769-byte proof, while a complete trade takes 34.6 s and contributes 7,264 bytes of effective on-chain payload. These measurements cover the prototype’s online execution and exclude secure preprocessing and a production setup ceremony.

II. NAVIGATING THE PRIVACY–DISCOVERY DILEMMA IN DECENTRALIZED EXCHANGES

An order book must publish enough to let the market form a price, and must publish little enough to keep its traders from being preyed upon. This section examines that dilemma: how an order book works and why exposure is costly (Section II-A), why existing countermeasures fail, and fail more severely on chain, to resolve it (Section II-B)—and what design point a privacy-preserving DEX should therefore aim at (Section II-C).

A. Order Books and the Anatomy of Parasitic Trading

a) *Limit order books:* A limit order book maintains the active buy and sell orders for a trading pair. An order is a tuple

$$o = (\text{id}, P, s, p, q_{\text{rem}}, t),$$

where id identifies the order, P the trader, $s \in \{\text{buy}, \text{sell}\}$ the side, p the limit price, q_{rem} the remaining unfilled quantity, and t the priority timestamp. The book state is $\text{LOB} = (\text{Bids}, \text{Asks})$, with Bids sorted by decreasing price and Asks by increasing price, ties broken by t . Crossing orders are matched under this price–time priority; a match determines the executed quantity and the resulting obligations, and settlement then transfers the assets and updates or removes the affected orders. A large trading interest typically exceeds the liquidity resting at any single price level, and is therefore worked as several orders across a band of levels, executing against a sequence of counterparties over time—which is what makes it visible for long enough to be exploited.

b) *The anatomy of parasitic trading:* Parasitic traders profit not from information about value but from other traders’ disclosed intentions. Suppose Alice works a large buying interest by posting several sizeable buy orders across a band of price levels. Because q_{rem} is public on every one of them, a parasitic trader [2], [3] reads off them that Alice has strong latent buying demand that will persist across those levels. The adversary accumulates in the same direction, taking the liquidity Alice would otherwise have obtained; the book she faces is thinned, so to obtain execution she must bid higher.

The adversary then supplies that liquidity back to her at the worse prices her own continued execution has produced. Alice bears an execution cost she would not have incurred had her size never been visible, and the adversary captures it.

Two properties of the attack generalize from this example, both concerning size rather than price. First, *the quantity is the signal*: side and limit price alone tell the adversary only that someone is willing to trade within that band, not the scale of the demand behind that willingness; it is q_{rem} that tells it whether the interest is worth attacking, how much inventory to commit, and how aggressively to move its own quotes. Second, *the attack has a size threshold*: positioning against Alice is not free—capital must be committed, quotes adjusted, liquidity provision forgone—and against orders of ordinary size these costs are not recovered, so the strategy is worthwhile only once the exposed quantity is large enough that the rent extracted exceeds the cost of extracting it.

Together these give the attack a necessary first step—*target selection*. Before positioning against anyone, the adversary must find a large resting order and distinguish it from the ordinary orders around it. This step is what the rest of the paper attacks: if no observer can tell a large order from a small one, no strategy conditioned on inferred size can outperform guessing, and the remainder of the pipeline has nothing to act on.

B. Conventional Countermeasures and Their Failure Modes

The defences markets already use all attack target selection, withholding in one way or another the size information it depends on; what each gives up follows from how much it withholds. We examine two. A third line of work delegates matching to a centralized matcher, which reinstates a discretionary intermediary and surrenders the decentralization.

a) *Withholding pre-trade information.*: The most direct response is to stop publishing order information altogether. Conventional dark pools do exactly this, as do fully hiding DEX designs, which conceal balances, sides, prices, and quantities alike. The cost is borne by price formation. Comerton-Forde and Putniņš observe that a limit order displayed in a transparent book can influence prices before it executes, whereas the information in a dark order cannot enter public prices until a trade occurs, and never does if the order goes unexecuted; they further find that a high level of non-block dark trading raises adverse-selection risk in the lit market and reduces informational efficiency, with the deterioration in their ASX(Australian Securities Exchange) sample beginning once dark trading in a stock exceeds roughly 10% of daily dollar volume [8]. For an order-book DEX whose market quality rests on public price formation, hiding the price dimension is therefore not a solution but a relocation of the damage.

b) *Partial display and statistical inference.*: Between full exposure and full darkness sits the iceberg order: only a peak is displayed while the remainder rests undisclosed, and as the peak executes a fresh tranche is replenished at the same price until the order is exhausted. Price and side stay public and continue to feed price formation, while the

quantity behind them does not. The concealment, however, is incomplete, and the leak is mechanical: replenishment is itself observable, and the arithmetic relating displayed to executed volume constrains what lies behind it. Hiding is therefore not the same as being unpredictable. Bessembinder finds that both the presence and the magnitude of hidden orders can be predicted to a significant, if imperfect, degree from observable order attributes, firm characteristics and market conditions [9], and detection algorithms operating directly on the book close the remaining gap: Zotikov and Antonov identify icebergs from inconsistencies between resting volume and the volume reported in trade summaries, together with the order modifications and fresh limit orders that follow a fill, then model the orders so identified to predict their total size, validated out of sample [12].

Concealment of this kind is therefore *obscurity rather than secrecy*: the quantity is never disclosed, yet remains estimable from the pattern the order’s own execution leaves in the public record, and as further fills and replenishments are observed the order leaves behind ever more execution traces from which its hidden size can be inferred—precisely on the large, long-lived orders that are worth attacking. On chain the position is worse still. The adversary’s raw material is the public record, and a blockchain supplies it in its most favourable form: the complete book history is permanently readable by anyone at no cost, at full granularity, replayable off-line and without the data-access asymmetries that limit who can run these inferences in a conventional venue. A defence whose strength is the empirical difficulty of an inference is weakest exactly where the inference is easiest to attempt.

C. Invisibook’s Design Point

The two subsections above bound the design space from opposite sides. Publishing the quantity hands target selection its input; hiding the price dimension protects the trader at the expense of the public price formation an order book exists to provide; and hiding the quantity by display policy alone yields a guarantee that degrades exactly on the orders that need it most. What the analysis leaves open is a design that concedes on neither axis: fully public along the price dimension, and protected along the size dimension by a guarantee that does not weaken as the order rests.

This is the design point Invisibook aims at. Price, side, trading pair and fee stay in plaintext, so matching remains an ordinary deterministic priority rule that the chain executes and every node verifies in public, and every order continues to contribute to price formation exactly as in a lit book. Quantity, by contrast, is hidden *cryptographically* rather than by display policy: quantities and balances are carried as commitments, so concealment rests on the hiding property of the commitment scheme rather than on the empirical difficulty of inverting an execution pattern—and, since nothing is displayed and replenished, no such pattern is left behind to invert. The guarantee is therefore uniform in order size, and strongest where the threat is: a large order is indistinguishable from a small one no matter how long it rests or how much of the

chain's history an adversary replays. The remainder of the paper realizes this trade-off: privacy along the size dimension, and preserved price discovery along the price dimension.

III. SECURITY MODEL

Building on the argument in Section II, the primary security objective of Invisibook is *amount privacy*. This objective covers both the quantity attached to an order and the balance amount controlled by each trader; the residual of a partially filled order remains an order quantity and is protected accordingly. We analyze this objective at two boundaries: the information traders submit to the public ledger and the private interaction between two matched traders during settlement.

A. Setting and Assumptions

Invisibook operates over a public ledger. Traders submit transactions and protocol messages that request ledger state transitions, while a matched pair may also interact over a point-to-point channel to compute a settlement. The ledger maintains the public protocol state and applies the validation rules prescribed by Invisibook. There is no separate exchange operator, matcher server, or trusted third party. This section abstracts from the concrete contents of these messages; Sections V and VI specify them.

Our guarantees are relative to the following properties of the underlying ledger:

- (i) *Consensus safety and finality*. Confirmed state transitions do not conflict and are not subsequently reversed.
- (ii) *Integrity of execution*. Every node applies the prescribed validation logic to the same transaction and ledger state.
- (iii) *Transaction inclusion*. A valid transaction submitted by an honest trader is eventually included and cannot be suppressed indefinitely.
- (iv) *Confidential balances*. The ledger provides a shielded asset representation that conceals balance amounts and prevents public linkage between spent and newly created outputs.

We additionally assume hiding and binding commitments, maliciously secure two-party computation with abort, a sound and zero-knowledge collaborative zk-SNARK, and unforgeable authentication of trader submissions. Off-chain traffic is carried over an anonymous communication channel when protection of the link between a network identity and an on-chain pseudonym is required. A failure of these primitives, the transport assumption, or any ledger property above lies outside the guarantees stated in this section. We assume neither trusted hardware nor a trusted operator.

B. Adversary Model

For security analysis, we consider two trader principals, A and B , and one public-observation principal, $\mathcal{O}$. The traders are the two parties that participate in an off-chain settlement interaction. The observer $\mathcal{O}$ represents the aggregate view available to blockchain nodes, parties maintaining the chain, other traders, and external observers. It is an analysis role rather than a third participant in the two-party settlement

protocol. All guarantees below quantify over arbitrary traders; no particular submitter is singled out in the model.

a) *Semi-honest public observer*: We model $\mathcal{O}$ as a semi-honest, or honest-but-curious, adversary. It follows the prescribed public protocol but records the complete ledger history and applies arbitrary PPT inference algorithms to that view. This restriction reflects the structure of the on-chain part of the system. On-chain operations are public and fixed by the protocol, their execution is deterministic, and every accepted state transition is revalidated by the ledger. The observer has no private step in the traders' two-party computation at which it can deviate. Moreover, an external trader that actively constructs a malicious submission is analyzed as a malicious trader rather than as $\mathcal{O}$. Deviations from consensus, execution integrity, or transaction inclusion are excluded by the ledger assumptions above. Thus $\mathcal{O}$ is passive with respect to execution but adversarial with respect to inference [14].

b) *Malicious trader*: Any trader acting as a submitter or as a settlement counterparty may instead be malicious. Such a trader may choose arbitrary inputs, send malformed or inconsistent messages, depart from the prescribed interaction, submit arbitrary data to the ledger, or abort. It controls its complete internal state and randomness and also observes everything available to $\mathcal{O}$. For an off-chain settlement between A and B , corruption is static and affects at most one of them. Security is required for either choice of the corrupted trader, rather than for an execution in which both traders are corrupted. This is the dishonest-majority two-party setting [14].

C. Security Properties

We state separate properties for the two security boundaries identified above. On chain, a malicious trader must not be able to induce an incorrect settlement, and the submissions of any honest trader must preserve its amount privacy against $\mathcal{O}$. Off chain, two honest traders must compute the prescribed result, while either trader must retain its amount privacy when the counterparty is malicious.

1) *On-Chain Settlement Submissions*: An *on-chain settlement submission* is any transaction or protocol message through which a trader asks the ledger to advance or finalize a settlement. The ordered sequence of all such submissions and the resulting public ledger states forms the on-chain settlement transcript. This abstraction covers every trader and every settlement without fixing the concrete message formats before the construction is presented.

Property 1 (Validity of on-chain settlement submissions). No PPT malicious trader can cause the ledger to accept a settlement submission whose resulting state transition violates the prescribed settlement relation, except with negligible probability. In particular, every accepted settlement must satisfy: (i) the quantity-dependent inputs are consistent with the commitments referenced by the settlement; (ii) the comparison result and filled quantity are computed correctly; (iii) every residual quantity is computed correctly and is non-negative; and (iv) the resulting balance updates are authorized and conserve value.

The property does not prevent a malicious trader from submitting arbitrary invalid data. It requires that such data be rejected before it can cause an incorrect settlement or ledger state transition.

Property 2 (On-chain amount privacy). For every honest trader T , the complete on-chain settlement transcript is computationally hiding with respect to T 's order quantities, including residual quantities, and its balance amounts. Equivalently, for any two executions that differ only in these amounts, the corresponding views of every PPT semi-honest observer $\mathcal{O}$ are computationally indistinguishable.

This property applies to every honest submitter and to the joint transcript of all its settlement submissions, rather than to a distinguished trader or to each message in isolation.

2) *Off-Chain Settlement Interaction*: An *off-chain settlement interaction* is the complete point-to-point protocol executed by a matched pair A, B to produce the result used by their subsequent settlement submissions. As above, the security model fixes the required properties without anticipating the concrete protocol messages.

Property 3 (Correctness of honest settlement). If A and B are honest and begin with valid private inputs, their off-chain interaction produces the settlement result prescribed for those inputs and the corresponding valid on-chain settlement submissions, except with negligible probability.

Property 4 (Privacy against a malicious counterparty). For either $i \in \{A, B\}$, suppose trader i is controlled by a PPT malicious adversary and trader j , $j \neq i$, is honest. The complete view of trader i during the off-chain settlement interaction is computationally hiding with respect to trader j 's order quantities and balance amounts. In particular, changing only these honest-party amounts yields computationally indistinguishable views for the malicious counterparty.

Property 5 (Scope under abort). A malicious counterparty may abort the off-chain interaction and thereby prevent it from completing. Such an abort must not violate the honest trader's amount privacy, and any invalid settlement submission produced before or after the abort remains subject to Property 1. We do not claim fairness or guaranteed output delivery.

IV. PRELIMINARIES

This section introduces the blockchain and cryptographic background used in our construction.

A. On-Chain States and Operations

A blockchain can be viewed as a public ledger that maintains on-chain states and validates state transitions. Decentralized exchanges are application-layer protocols built on top of this ledger infrastructure, using on-chain operations to represent, update, and settle trading states.

1) *UTXO Model*.: The following UTXO and transaction definitions adapt the formal Bitcoin transaction model of Atzei et al. [15] to a multi-asset ledger with explicit spending predicates. The UTXO model consists of UTXOs and transactions.

A *UTXO* is an unspent transaction output that represents a spendable asset together with its spending condition, formally denoted by

$$u = (r, a, v, \phi),$$

where $r = (\text{txid}, j)$ is the output reference, indicating that this UTXO is the j -th output created by transaction txid; $a \in \mathcal{A}$ is the asset type; $v \in \mathbb{V}$ is the asset value; and $\phi \in \Phi$ is the spending predicate specifying the condition under which the output can be consumed.

A transaction consumes existing UTXOs as inputs and creates new UTXOs as outputs, defined as follows:

$$T = (\text{in}, \text{wit}, \text{out}),$$

where $\text{in} = (r_1, \dots, r_m)$ is a sequence of input references to existing UTXOs, $\text{wit} = (w_1, \dots, w_m)$ is a sequence of witnesses, and $\text{out} = ((r'_1, a'_1, v'_1, \phi'_1), \dots, (r'_n, a'_n, v'_n, \phi'_n))$ is a sequence of newly created outputs. Each output in out obtains a fresh reference $r'_j = (H(T), j)$, where H is a collision-resistant hash function.

Given a ledger state L , a transaction T is valid if the following conditions hold:

- 1) Every input reference r_i refers to an unspent output.

$$u_i = (r_i, a_i, v_i, \phi_i) \in \text{UTXO}(L).$$

- 2) Each witness satisfies the corresponding spending predicate.

$$\phi_i(T, w_i) = 1 \quad \text{for all } i \in [m].$$

- 3) Value is conserved for each asset type. That is, for every $a \in \mathcal{A}$,

$$\sum_{i: a_i = a} v_i \geq \sum_{j: a'_j = a} v'_j.$$

The difference between the two sides represents transaction fees or burned value.

A valid transaction updates the ledger state by removing the consumed outputs and adding the newly created outputs:

$$\text{UTXO}(L') =$$

$$\text{UTXO}(L) \setminus \{(r_i, a_i, v_i, \phi_i)\}_{i=1}^m \cup \{(r'_j, a'_j, v'_j, \phi'_j)\}_{j=1}^n.$$

2) *Decentralized Exchanges*: A decentralized exchange (DEX) is a distributed-ledger-based protocol or application that enables users to trade crypto assets without relying on a centralized exchange operator as either the trading intermediary or asset custodian [1]. At a high level, a DEX defines application-layer trading states and operations on top of the underlying ledger. These states are ultimately represented by ledger states, and the operations are implemented as ledger transactions that update those states.

B. Cryptography

Let λ denote the security parameter, and let $\text{negl}(\lambda)$ denote a negligible function. Two distribution ensembles $\{X_\lambda\}_\lambda$ and $\{Y_\lambda\}_\lambda$ are *computationally indistinguishable*, written $X \approx_c Y$, if for every probabilistic polynomial-time (PPT) distinguisher D ,

$$|\Pr[D(X_\lambda) = 1] - \Pr[D(Y_\lambda) = 1]| \leq \text{negl}(\lambda).$$

This is the notion of indistinguishability used throughout this section [16].

1) *Maliciously Secure MPC*: Secure multiparty computation allows parties with private inputs to compute a prescribed functionality while revealing no more than their prescribed outputs. A *semi-honest* adversary corrupts a party and observes its entire internal state, but the corrupted party follows the protocol. A *malicious* adversary may instead make a corrupted party deviate arbitrarily, including by changing its effective input, sending arbitrary messages, or aborting [14].

Definition 1 (Maliciously secure two-party computation). Let Π be a two-party protocol intended to realize an ideal functionality $\mathcal{F}$. Denote by $\text{REAL}_{\Pi, \mathcal{A}}(\lambda, \mathbf{x}, z)$ the joint output of the parties and a PPT real-world adversary $\mathcal{A}$ on inputs $\mathbf{x} = (x_1, x_2)$ and auxiliary input z . Denote by $\text{IDEAL}_{\mathcal{F}, \mathcal{S}}(\lambda, \mathbf{x}, z)$ the corresponding joint output in an ideal execution, where a PPT simulator $\mathcal{S}$ controls the same corrupted party and interacts with $\mathcal{F}$. The protocol Π securely realizes $\mathcal{F}$ against a static malicious adversary corrupting at most one party if, for every PPT $\mathcal{A}$, there exists a PPT $\mathcal{S}$ such that, for all $\mathbf{x}$ and z ,

$$\text{REAL}_{\Pi, \mathcal{A}}(\lambda, \mathbf{x}, z) \approx_c \text{IDEAL}_{\mathcal{F}, \mathcal{S}}(\lambda, \mathbf{x}, z).$$

In the security-with-abort formulation, the simulator may cause the ideal execution to abort; consequently, this definition does not by itself provide fairness or guaranteed output delivery [14].

2) *zk-SNARKs*: Let $\mathcal{R}$ be an NP relation, and let $L_{\mathcal{R}} = \{x : \exists w \text{ such that } (x, w) \in \mathcal{R}\}$. Following the standard definitions of zero knowledge and succinct non-interactive arguments [17], [18], we use the following interface.

Definition 2 (zk-SNARK). A zk-SNARK for $\mathcal{R}$ consists of three PPT algorithms:

- $\text{Setup}(1^\lambda, \mathcal{R}) \rightarrow \text{pp}$ generates public parameters;
- $\text{Prove}(\text{pp}, x, w) \rightarrow \pi$ generates a proof for a statement–witness pair (x, w) ; and
- $\text{Verify}(\text{pp}, x, \pi) \rightarrow \{0, 1\}$ accepts or rejects the proof.

It satisfies the following properties:

- *Completeness*. For every $(x, w) \in \mathcal{R}$, an honestly generated proof is accepted except with negligible probability.
- *Soundness*. No PPT prover can make the verifier accept a statement $x \notin L_{\mathcal{R}}$ except with negligible probability. For an argument of knowledge, any PPT prover that produces an accepting proof with non-negligible probability admits a PPT extractor that recovers a witness w such that $(x, w) \in \mathcal{R}$.

- *Zero knowledge*. There exists a PPT simulator that, given the public statement but not the witness, produces public parameters and a proof computationally indistinguishable from an honest setup and proof.
- *Non-interactivity*. After setup, the prover sends one proof π and receives no message from the verifier.
- *Succinctness*. The proof size and verification time are polynomial in λ , the public-input length, and the logarithm of the relation size, rather than in the full witness-computation size.

3) *Collaborative zk-SNARKs*: A standard zk-SNARK assumes that one prover holds the entire witness. When the witness is distributed among mutually distrustful parties, a *collaborative zk-SNARK* replaces the prover algorithm with a protocol among the witness holders [13].

Definition 3 (Collaborative Zero-Knowledge Succinct Non-interactive Argument of Knowledge). Let $(\text{Setup}, \text{Prove}, \text{Verify})$ be a zk-SNARK for an NP relation $\mathcal{R}$. A collaborative zk-SNARK for N provers consists of $(\text{Setup}, \Pi, \text{Verify})$, where Setup and Verify are those of the underlying zk-SNARK and

$$\Pi(\text{pp}, x, w_1, \dots, w_N) \rightarrow \pi$$

is a protocol that evaluates the underlying prover algorithm on the distributed witness $(w_1, \dots, w_N)$ and outputs one proof π .

In addition to the underlying zk-SNARK properties, the protocol is *t-zero-knowledge* if, for every PPT adversary $\mathcal{A}$ controlling a set Corr of at most t provers, there exists a PPT simulator $\mathcal{S}$ such that

$$\text{View}_{\mathcal{A}}^{\Pi}(\text{pp}, x, w_1, \dots, w_N) \approx_c \mathcal{S}(\text{pp}, x, b, \{w_i\}_{i \in \text{Corr}}), \quad (1)$$

where $b \in \{0, 1\}$ is one exactly when $(x, w_1, \dots, w_N) \in \mathcal{R}$. Thus the corrupted provers' view reveals no additional information about the honest provers' witnesses beyond the public statement, the relation result, and the corrupted provers' own inputs [13].

In our application, $N = 2$, and the collaborative proving protocol Π is instantiated using maliciously secure two-party MPC. The concrete protocol is specified in Section VI; merely using MPC to execute the prover does not replace the *t-zero-knowledge* requirement above [19].

4) *Commitment*:

a) *Commitment*.: A setup-based non-interactive commitment scheme [20] is a pair of randomized algorithms $\text{Com} = (\text{Setup}, \text{Commit})$. The setup algorithm $\text{Setup}(1^\lambda)$ outputs public parameters pp , and the commitment algorithm $\text{Commit}(\text{pp}, m; r)$ outputs a commitment c to message m using randomness r . An opening (m, r) is accepted if $c = \text{Commit}(\text{pp}, m; r)$.

A commitment scheme satisfies two standard security properties.

- *Hiding* requires that the commitment c reveals no information about m : for any two messages m_0, m_1 , no PPT adversary can distinguish a commitment to m_0 from a commitment to m_1 except with negligible advantage.

- *Binding* requires that no PPT adversary can find a commitment c that admits two valid openings (m_0, r_0) and (m_1, r_1) with $m_0 \neq m_1$, except with negligible probability.

In Sections V and VI, we treat the above cryptographic mechanisms as black-box functionalities and use them to describe the protocol independently of a concrete implementation. We defer concrete instantiation and evaluation to Section VII.

5) *Digital Signature*: A digital signature scheme [21] consists of three algorithms: Gen generates a public/secret key pair, Sign produces a signature on a message using the secret key, and Vrfy checks the signature using the public key. Its standard unforgeability requirement is that no PPT adversary with access to signatures on adaptively chosen messages can produce a valid signature on a new message except with negligible probability.

V. ON-CHAIN STATE AND OPERATIONS

This section specifies the on-chain data structures of Invisibook and the life cycle of a trade as observed by the blockchain: order submission, matching, and the verification of settlement results. The off-chain settlement protocol is only summarized here at the interface level; its full construction is deferred to Section VI. Throughout, we write $\text{Com}(x; r)$ for a commitment to the message x (possibly a tuple) under randomness r , and $[P]$ for the truth value of predicate P .

A. Amount-Hiding UTXO

Invisibook adopts a UTXO-based account model in which, exactly as in Zcash, *nothing* about a UTXO appears in plaintext on-chain. Formally, the content of a UTXO is a tuple

$$\text{utxo} = (\text{pk}, \text{assetID}, v), \quad \text{cm} = \text{Com}(\text{utxo}; r),$$

where pk is the owner's public key, assetID identifies the asset type, and v is the amount. The chain records only the hiding and binding commitment cm : neither the owner's key, nor the asset type, nor the amount is ever published. Ownership and asset-type validity are instead verified in zero-knowledge at spending time, and binding assetID inside the commitment ties each amount to its asset type, so that a proof over committed amounts of one asset can never be reused for or mixed with another. Only the owner knows the opening (utxo, r) . The randomness r (a blinding factor) is indispensable: amounts live in a small, enumerable space, so without r any observer who knows or guesses pk and assetID could precompute the commitments of all candidate amounts and recover v by lookup—a rainbow-table-style dictionary attack.

Our UTXO definition deliberately mirrors that of Zcash [22], [23], and we refer to these as *private UTXOs*: commitments are accumulated in an append-only Merkle tree, and a UTXO is consumed by publishing a *nullifier* that is cryptographically bound to the note but unlinkable to its commitment. Spending is authorized by a zero-knowledge proof of (i) membership of the note commitment in the Merkle tree, (ii) knowledge of the spending key, and (iii)

balance preservation over committed amounts. The privacy guarantees of this construction—amount confidentiality via commitment hiding and spend/output unlinkability via nullifiers—have been formally established in the Zerocash security analysis [22]; Invisibook inherits these guarantees directly and we therefore do not re-prove them. In particular, they ensure that no on-chain observer can trace or analyze any individual UTXO—large UTXOs included—which is precisely the property we need: it denies the attacker the ability to locate large holdings at the UTXO level.

We stress that using Zcash to develop the account-layer privacy in this section is only for concreteness and ease of exposition. The requirement Invisibook places on the underlying blockchain (stated earlier) is that account balances must be hidden; Zcash is merely one construction that meets it. Any protocol achieving the same effect—hiding amounts while keeping UTXOs untraceable and unlinkable—is equally applicable, and Invisibook is not tied to Zcash.

B. Amount-Hiding Order

An order is a *special UTXO*: unlike an ordinary private UTXO, whose entire content is hidden inside its commitment, an order deliberately publishes every attribute that public matching requires, hiding only its quantity. Formally, an order is a tuple

$$o = (\text{oid}, \tau, d, p, \text{cm}_q, \text{pk}, f), \quad \text{cm}_q = \text{Com}(q; r_q),$$

where oid is a unique order identifier, τ the trading pair, $d \in \{\text{buy}, \text{sell}\}$ the side, p the limit price *in plaintext* (or a market-order flag), cm_q a commitment to the order quantity q (with r_q blinding the commitment exactly as r does for notes), pk the owner's key—published so that the chain can authenticate the owner's subsequent messages, namely its proof share and its settlement update—and f the *plaintext* fee carried by the order. The fee is denominated in the chain's native token and its amount is public by design, since on-chain matching must consult fee sizes (see the priority rules below). The block height at which the order transaction is included and the transaction's index within its block are properties of the enclosing transaction rather than of the order itself; as public chain metadata they nevertheless participate in matching priority, as described below.

Creating an order consumes private UTXOs of total value $p \cdot q + f$. Exactly as in Zcash, the submitter destroys the input UTXOs by publishing their nullifiers—the nullifier mechanism is what guarantees that the spent UTXOs cannot be traced or analyzed—and generates the corresponding zero-knowledge proof and metadata, attesting that the committed total of the destroyed UTXOs equals the committed value $p \cdot q$ plus the plaintext fee f ; that is, the order is fully collateralized. The fee is paid to the miner in the same way transaction fees are handled in Zcash: the fee amount is public while every other value remains hidden. Only after the chain verifies this spend is o admitted into the order book; thus the book never contains uncollateralized orders, even though no verifier learns any quantity.

C. Trade Flow

a) Matching.: Since prices are public, matching requires no cryptographic machinery: the chain runs a conventional deterministic price–time priority algorithm directly over the plaintext price field. Concretely, crossing orders are matched under the following priority rules:

- 1) *Price priority.* Among crossing orders, the best-priced order is matched first.
- 2) *Time priority.* Ties in price are broken by the block height at which the order transaction was included: orders in earlier blocks are matched first.
- 3) *Fee priority.* If tied orders share both price and block height, they are matched in descending order of the plaintext fee f carried by the order.
- 4) *Intra-block index.* If price, block height, and fee all coincide, the order whose transaction has the smaller `transaction_index` within the block is matched first.

A structural consequence of amount hiding is that matching is strictly *pairwise*: each match consists of exactly one bid and one ask. In a transparent order book, the matching engine fills a large incoming order by walking the book and aggregating multiple resting orders until the incoming quantity is exhausted. This is impossible here—the chain cannot directly compare, split, or sum quantities it only sees as commitments, and so it cannot even determine *when* an incoming order is exhausted. A match in Invisibook is therefore a *price match* rather than a full-fill match: the chain pairs the incoming order with the single highest-priority crossing order, and all quantity-dependent computation is deferred to settlement. Once a pair is matched, the two orders are immediately locked on-chain: they can no longer be canceled and are forced into the settlement process.

b) Off-chain comparison and on-chain verification.: Settlement begins by privately confirming which of the two orders has the smaller quantity. The matched parties A and B execute a secure multi-party computation off-chain (Section VI) which, entirely inside the protocol, (i) verifies that each party’s private input opens its on-chain quantity commitment, and (ii) compares the two quantities. The computation reveals nothing beyond a single bit $b = [q_A \leq q_B]$ identifying the smaller order (if the quantities are equal, both orders are fully filled). Alongside b , a zero-knowledge proof is collaboratively generated: each party submits its own share of the co-ZK proof to the chain, where the shares are assembled into a single complete co-ZK proof π_{cmp} (the construction of the shares and of their assembly is given in Section VI), attesting that b is the correct comparison of the values committed in cm_{q_A} and cm_{q_B} . The chain then verifies π_{cmp} against the locked orders’ commitments. Only after this on-chain verification succeeds does the protocol proceed.

c) State updates.: Suppose without loss of generality that A holds the smaller order. Once the comparison has been verified on-chain, the parties complete the remaining interaction of the off-chain settlement protocol—through which the larger

side learns the fill quantity; how is specified in Section VI—and each then submits its own state update to the chain:

- *Smaller side (A).* A ’s order, being fully filled, is *marked as destroyed* directly on the order book. Orders are public book entries, so this destruction is an explicit public marking and does *not* go through the nullifier mechanism. Alongside the destruction, a new private UTXO is created and paid to the counterparty B —a fresh note commitment whose committed amount corresponds to executing the full committed quantity q_A at the match price p —together with a proof π_A of this correspondence.
- *Larger side (B).* B ’s order is likewise *marked as destroyed*, and two objects are created in its place: a *new order* o'_B carrying the same pair, side, and price, whose quantity commitment $\text{cm}'_{q_B} = \text{Com}(q_B - q_A; r'_B)$ commits to the residual quantity under fresh randomness r'_B , and a new private UTXO paid to the counterparty A . These are accompanied by a proof π_B attesting that (i) the residual equals $q_B - q_A$ and is non-negative, (ii) the q_A used in this computation opens A ’s on-chain commitment cm_{q_A} —so B cannot understate the fill by substituting a fabricated counterparty quantity—and (iii) the committed amount of the new UTXO is consistent with the fill.

The chain verifies each proof against the current on-chain commitments and, upon success, applies the updates: both original orders are destroyed on the book, the residual order o'_B enters the book, and the newly created UTXOs join the private UTXO set as ordinary note commitments, indistinguishable from any other note. Any later spend of these UTXOs goes through the standard Zcash-style nullifier mechanism, so they cannot be traced or analyzed in subsequent operations.

At no point does the chain—or any observer—learn a plaintext quantity: matching operates on public prices, and settlement verification operates on commitments and a single comparison bit whose disclosure is inherent to order-book semantics (the destruction marking of a fully filled order is publicly observable regardless).

VI. OFFCHAIN PRIVACY SETTLEMENT

Matching only produces a *price match*: because order sizes are hidden behind commitments, the on-chain matching engine cannot tell whether the two matched quantities are equal, nor which is larger. The actual size comparison, the residual computation, and the state update are therefore deferred to an off-chain privacy-preserving settlement between the two matched parties, whom we denote A and B (Alice and Bob). Settlement must transition both parties’ private states and be publicly verifiable on chain, while revealing to neither party—nor to any observer—more about the counterparty’s order size than the settlement outcome inherently requires.

The core difficulty is the size comparison. Each order size is recorded on chain as a hiding and binding commitment $\text{cm}_q = \text{Com}(q, r)$, where q is the plaintext quantity and r a blinding factor, both known only to the order’s owner. Neither party may reveal its plaintext quantity to the other in order to compare, since that would leak precisely the private value

settlement is meant to protect. Indeed, probing the counterparty's quantity during settlement is the central adversarial behavior the threat model of Section III-B must capture. We resolve this with a maliciously secure two-party computation that compares the two committed quantities without either party learning the other's input.

We stress that the two computing parties are *the traders themselves*: the matched pair A and B jointly run the secure comparison, each supplying its own private order data as input. There is no external server, no designated computation service, and no trusted third party. Consequently the party against whom each trader must be protected is its own co-computing counterparty. This mirrors the setting of collaborative zero-knowledge proofs [13], in which a proof over a distributed witness is produced jointly by the witness holders without any of them learning the others' shares; here the distributed witness is the pair of committed order sizes, and the secure computation is instantiated with SPDZ-style secret-sharing MPC [24].

A. Secure Comparison

The first phase of settlement determines which order is smaller. Let q_A, r_A (resp. q_B, r_B) be A 's (resp. B 's) plaintext quantity and blinding factor, and let cm_{q_A}, cm_{q_B} be the corresponding on-chain commitments. A and B run a two-party MPC that, entirely under secret sharing, evaluates the comparison

$$b = \text{sign}(q_A - q_B) \in \{-1, 0, 1\}, \quad (2)$$

where $b = -1$ means $q_A < q_B$ (A is the smaller order), $b = 0$ means $q_A = q_B$ (the two orders are equal), and $b = 1$ means $q_A > q_B$ (B is the smaller order).

The comparison alone says nothing about what the parties actually fed into the computation. A malicious party could supply a fabricated quantity and thereby manipulate b . We therefore use a collaborative zero-knowledge proof (co-ZK) [13] to guarantee the *legitimacy of the MPC inputs*: to prove that the inputs to the computation are exactly the order quantities the parties previously committed to on chain. Concretely, the two parties *jointly generate*, inside the MPC, a publicly verifiable zero-knowledge proof π_{cmp} . Let share_A^b and share_B^b be the two additive shares of the comparison result b output by the MPC, and let $cm_A^b = \text{Com}(\text{share}_A^b, \rho_A)$ and $cm_B^b = \text{Com}(\text{share}_B^b, \rho_B)$ be commitments to those shares, where ρ_A and ρ_B are blinding factors. The statement of π_{cmp} is that there exist (q_A, r_A) , (q_B, r_B) , b , $(\text{share}_A^b, \rho_A)$ and $(\text{share}_B^b, \rho_B)$ such that

$$\pi_{\text{cmp}} : \begin{cases} \text{Com}(q_A, r_A) = cm_{q_A}, \\ \text{Com}(q_B, r_B) = cm_{q_B}, \\ b = \text{sign}(q_A - q_B), \\ \text{share}_A^b + \text{share}_B^b \equiv b \pmod{p}, \\ cm_A^b = \text{Com}(\text{share}_A^b, \rho_A), \\ cm_B^b = \text{Com}(\text{share}_B^b, \rho_B). \end{cases} \quad (3)$$

Its public inputs are $cm_{q_A}, cm_{q_B}, cm_A^b, cm_B^b$; the comparison result b itself and both shares remain witnesses and are not disclosed in the first step.

The first three constraints capture *input legitimacy and correctness of the comparison*—clause (i) of Property 1: the values compared are exactly the quantities that open the on-chain commitments cm_{q_A}, cm_{q_B} , and b is the correct result of comparing them. The last three *pin down the shares*: the two shares really do sum to that legitimate b , and they open the public commitments cm_A^b and cm_B^b respectively. Because π_{cmp} is produced jointly by the two parties inside the MPC, with neither learning the other's witness share, it simultaneously anchors the MPC inputs to the on-chain commitments and fixes in advance the shares either party may submit in the second step—without revealing q_A, q_B , or even b —rendering both guarantees publicly verifiable by the chain and every third party.

The MPC output is not revealed to either party in the clear; it is produced as additive shares—of both the comparison result b and the jointly generated proof π_{cmp} . Party A holds share share_A^b and share_A^{π} , party B holds share_B^b and share_B^{π} , where

$$\text{share}_A^b + \text{share}_B^b \equiv b \pmod{p},$$

and share_A^{π} together with share_B^{π} combine to the complete π_{cmp} . Crucially, both kinds of share *must be submitted on chain*: the parties reconstruct neither of them between themselves, and both b and π_{cmp} are assembled only on chain, in two successive steps (Section VI-B). Until then, what each party holds is an uninformative share.

We use SPDZ [24], a maliciously secure MPC framework built on additive secret sharing and information-theoretic MACs (IT-MACs), as the secure computation underlying the co-ZK. The IT-MACs guarantee that if either party tampers with its input or with any intermediate value, the deviation is detected by the other party and the protocol aborts before any value is opened.

B. On-Chain Share Combination

Submission and assembly proceed in two steps whose order cannot be reversed: the proof is assembled and verified first, and only then is the comparison result assembled. Together these two steps discharge clause (ii) of Property 1—that the computation producing b is itself correct and untampered, and that neither party can unilaterally spoil its output.

Step 1: submit proof shares and share commitments; assemble and verify π_{cmp} . Within Δ_{sub} block times ($\Delta_{\text{sub}} = 10$ in our instantiation), each party submits two items: its proof share, and the commitment to its own share of the comparison result—that is, A submits $(\text{share}_A^{\pi}, cm_A^b)$ and B submits $(\text{share}_B^{\pi}, cm_B^b)$. These commitments are public inputs of π_{cmp} and are *not* contained in the proof shares, so they must be supplied explicitly alongside the shares for the chain to verify the proof at all.

The chain first assembles the two proof shares into the complete proof

$$\pi_{\text{cmp}} \leftarrow (\text{share}_A^{\pi}, \text{share}_B^{\pi}),$$

and then verifies π_{cmp} against the public inputs $\text{cm}_{q_A}, \text{cm}_{q_B}, \text{cm}_A^b, \text{cm}_B^b$. A successful verification establishes two things at once: that the compared inputs are the legitimate quantities opening $\text{cm}_{q_A}, \text{cm}_{q_B}$, and that the two share commitments are now fixed on chain as the reference against which the shares are checked in the second step. No trust is needed in the submitted commitments: a proof is generated for one specific set of public inputs, so any substituted cm_i^b makes verification fail immediately. If verification fails, the settlement terminates here: the shares of the comparison result are never submitted, and b is never reconstructed.

Step 2: submit the shares and assemble b . Only after the on-chain verification of Step 1 succeeds do the parties submit their shares of the comparison result, within a further Δ_{sub} block times. Crucially, each party must *accompany* its share with a zero-knowledge proof that the submitted share is indeed the result of the very computation attested by the already-verified π_{cmp} : A submits $(\text{share}_A^b, \pi_{\text{bind}}^A)$ and B submits $(\text{share}_B^b, \pi_{\text{bind}}^B)$, where

$$\pi_{\text{bind}}^i : \exists \rho_i \text{ s.t. } \text{cm}_i^b = \text{Com}(\text{share}_i^b, \rho_i), \quad i \in \{A, B\}. \quad (4)$$

That is, the submitted share does open the commitment cm_i^b that was fixed on chain alongside π_{cmp} in Step 1. The chain verifies π_{bind}^A and π_{bind}^B , and only once both pass does it add the shares,

$$b = \text{share}_A^b + \text{share}_B^b \bmod p,$$

and publish b , which triggers the remainder of settlement.

This ordering, together with the binding proof, is deliberate, and its necessity follows from a concrete attack. Suppose A is malicious and B honest. If both kinds of share were submitted together and the chain simply added them to recover b , A could submit a forged share share_A^b . Being honest, B submits its true share share_B^b , which becomes visible to A the moment it appears on chain; A then adds it to the true share it actually holds and recovers the *real* b locally. The chain, meanwhile, adds B 's true share to A 's forged one and obtains a wrong b . The outcome is an asymmetry entirely in the adversary's favour: A alone learns the true comparison result, while the public on-chain result has been unilaterally corrupted—precisely the failure clause (ii) of Property 1 rules out.

The two-step ordering together with π_{bind} closes this path. When π_{cmp} is verified in Step 1, the share commitments $\text{cm}_A^b, \text{cm}_B^b$ are already fixed on chain as its public inputs, and the proof guarantees that the shares they commit to do sum to the legitimate b . In Step 2, a party wishing to submit a forged share would have to produce a verifying π_{bind} for that forged value—equivalent to forging a commitment opening for a value it never committed to—which, by the binding property of the commitment scheme and the soundness of the proof system, is computationally infeasible. A forged share is therefore rejected outright on chain rather than silently combining into an incorrect b .

Assembling on chain rather than reconstructing between the parties makes the finality of b and π_{cmp} independent of

either party: neither can unilaterally determine or forge the outcome. If a party fails to submit the corresponding share within the deadline of either step, the liveness mechanism of Section VI-D applies.

C. Settlement

Once b is public, the two parties settle asymmetrically according to who holds the smaller order. Without loss of generality assume $b = -1$, so A is the smaller order and B the larger; the case $b = 1$ is symmetric with the roles exchanged, and the case $b = 0$ is treated below.

Smaller party (A): reveal and finalize. A sends its plaintext quantity q_A and blinding factor r_A to B over the secure point-to-point channel. This is the disclosure the outcome inherently requires: the traded amount equals the smaller order's full quantity, so once A is known to be smaller, B must learn q_A to settle. A then updates its on-chain state, marking its order *fully filled* and submitting the shielded-balance update reflecting the settled q_A , following the on-chain state update procedure of Section V-C.

Larger party (B): verify, compute residual, re-commit. On receiving (q_A, r_A) , B first checks locally that

$$\text{Com}(q_A, r_A) = \text{cm}_{q_A},$$

confirming that the plaintext A sent matches A 's on-chain commitment and that A is not cheating. B then computes its residual quantity

$$q'_B = q_B - q_A,$$

samples a fresh blinding factor r'_B , and forms a new order commitment $\text{cm}'_{q_B} = \text{Com}(q'_B, r'_B)$, so that the residual never appears in plaintext at any point. Finally B submits to chain: the updated order commitment $\text{cm}_{q_B} \rightarrow \text{cm}'_{q_B}$ (the residual order remains resting in the book), the corresponding shielded-balance update following the same procedure, and a zero-knowledge proof π_B attesting

$$\pi_B : \begin{cases} q'_B = q_B - q_A, & (\text{residual computed correctly}) \\ \text{Com}(q_A, r_A) = \text{cm}_{q_A}. & (q_A \text{ opens } A\text{'s commitment}) \end{cases} \quad (5)$$

Statement (1) proves B did not tamper with the residual arithmetic; statement (2) proves that the q_A used by B is exactly the value A committed to on chain, preventing B from fabricating a smaller counterparty quantity to its own advantage. Together with the balance-update proofs of Section V-C, these discharge clauses (iii) and (iv) of Property 1; the non-negativity of the residual required by clause (iii) follows from the publicly verified b , which already certifies $q_B \geq q_A$.

Equal quantities ($b = 0$). When $q_A = q_B$, both orders are fully filled: the residual $q'_B = 0$, B 's order is likewise marked fully filled, and each party can already infer that the counterparty's quantity equals its own.

The disclosure here is deliberate and minimal, and matches Property 4. The larger party learns the smaller party's quantity, which is exactly the traded amount—the minimum any settlement functionality must reveal—while the smaller party learns

only that the counterparty’s quantity exceeds its own, exactly the leakage already implied by the public value b . Neither the chain nor any third party learns either quantity.

D. Liveness and the Challenge Mechanism

This subsection specifies the mechanism realizing Property 5. Our protocol provides security with abort: a malicious counterparty cannot violate correctness or privacy, but it can stall settlement—not by deviating detectably, but by simply withholding messages. Since a matched pair is locked while settlement is pending, withholding would otherwise let a malicious party freeze the counterparty’s order and capital indefinitely at no cost to itself. The protocol exposes two natural withholding points, each backstopped by an on-chain timeout. Not every abort, however, warrants a penalty: what calls for an asymmetric penalty is the deliberate withholding of a result already computed.

The comparison phase: aborting mid-computation versus withholding a computed result. An abort in the comparison phase falls into two cases whose consequences differ sharply.

Case 1: aborting during the MPC. A malicious party may unilaterally terminate the protocol before the secure computation completes. Because the underlying MPC is maliciously secure, and because b and π_{cmp} are produced—in shared form—only upon completion, a party that aborts midway *learns nothing*: its view is simulatable, so it cannot infer the counterparty’s order quantity from the messages exchanged, and it is likewise *unable to produce* a valid share of π_{cmp} or of b . Neither party then holds a submittable result, so no shares reach the chain at all. After the timeout, the chain returns both orders to the order book to re-enter matching, with no penalty imposed on either party. The aborting party gains nothing from such an abort—it has obtained no information and inflicted no additional loss on its counterparty; the only consequence is that both sides have wasted one round of matching.

Case 2: withholding a computed result. Only once the MPC has completed, and both parties hold their shares of b and π_{cmp} , does unilateral withholding become a meaningful attack: at that point the compliant party has already paid the computational cost while its order remains locked. Each of the two submission steps therefore carries its own deadline of Δ_{sub} block times ($\Delta_{\text{sub}} = 10$ in our instantiation): the proof shares in the first step, and the comparison-result shares in the second, once the proof has verified. If exactly one party has submitted by the deadline of either step, the silent party is treated on chain as the delaying party: its order—and the assets that order has locked—is frozen for a penalty period (72 hours in our instantiation), while the compliant party’s order is released back to the book and may immediately re-enter matching.

Withholding the plaintext at settlement. After b is published, the smaller party must send the opening (q, r) of its commitment to the larger party over the point-to-point channel. This step is off chain and hence not directly observable on chain, so it is enforced by a challenge. If the smaller party

(say A) does not send the plaintext within Δ_{reveal} block times ($\Delta_{\text{reveal}} = 5$), the larger party B files an on-chain enforcement request carrying a public key pk_B of its choosing, compelling A to post $\text{Enc}_{\text{pk}_B}(q_A, r_A)$ on chain. A must respond within Δ_{resp} block times ($\Delta_{\text{resp}} = 10$); failing that, A ’s order—and its locked assets—is frozen for the penalty period and B ’s order is released for re-matching. Upon a timely response, B downloads the ciphertext and decrypts it locally to recover (q_A, r_A) , then checks whether $\text{Com}(q_A, r_A)$ equals A ’s on-chain commitment cm_{q_A} . If it does, B completes verification and settlement exactly as above. If it does not—that is, A has posted a plaintext inconsistent with its own commitment— B posts the offending plaintext (q_A, r_A) on chain and initiates adjudication. On-chain logic then checks, in order: (1) that the plaintext indeed originated from A , by re-encrypting it under B ’s public key pk_B and comparing against the ciphertext $\text{Enc}_{\text{pk}_B}(q_A, r_A)$ that A previously posted; and (2) if it did originate from A , whether $\text{Com}(q_A, r_A) = \text{cm}_{q_A}$ holds. If check (1) passes but check (2) fails, A is proven to have submitted a false plaintext: A ’s order is frozen for the penalty period and B ’s order is released for re-matching.

Routing the response through the chain serves two purposes. First, it makes the disclosure publicly attributable and adjudicable: either a valid response exists on chain by the deadline or it does not, and if a false plaintext appears, the two checks above let the chain assign blame unambiguously—all without the honest party ever revealing its private key. Second, on the honest path it preserves privacy: by the semantic security of the encryption scheme, the posted ciphertext reveals nothing to third parties, and its plaintext content is precisely the disclosure the larger party is already entitled to receive. (The adjudication path does expose (q_A, r_A) on chain, but only once A is proven to have cheated; at that point A has forfeited any legitimate claim to privacy, and its true quantity would in any case have been revealed to B by settlement.)

The penalty is deliberately asymmetric—the staller is frozen while the victim is freed—which makes withholding strictly unprofitable: the withholding party forfeits market access for the entire penalty period, whereas the honest party’s cost is bounded by the timeout delay before its order re-enters the book.

The *form* and *duration* of the penalty are equally deliberate. We freeze the offending party’s order, and the assets it has locked, for a period (typically several days) rather than slashing its funds. We do not slash because a missed response is not necessarily malicious: the counterparty may genuinely have missed the deadline due to a network fault or disconnection, and confiscating assets would be an unduly harsh response to such cases. The point of a temporary freeze is instead to make the delaying party *miss the prevailing market conditions and trading opportunities*: while its order is frozen, the released honest trader is free to complete its trades under the current market. The deterrent thus derives from *opportunity cost* rather than loss of principal—for a parasitic trader whose aim is to lock up a counterparty’s capital and wait for an advantageous moment, forfeiting the market window

alone renders the strategy unprofitable, while for an honest party merely knocked offline the cost is bounded to missing one window of market activity rather than losing assets.

VII. IMPLEMENTATION AND EVALUATION

We implemented an end-to-end prototype of Invisibook and evaluate three aspects of its cost: the overhead of collaborative proof generation, its sensitivity to network latency, and the latency and on-chain cost of a complete trade. The evaluation reports measurements of the implemented system; it does not claim throughput or deployment-scale performance beyond the evaluated workloads.

A. Implementation and Experimental Methodology

The protocol is implemented primarily in Rust. The two-party computation layer builds on a pinned fork of `ark-mpc`, which provides SPDZ-style authenticated secret sharing, MAC checking, Beaver multiplication, and QUIC communication [24]. Collaborative proof generation uses a pinned fork of `mpc-jellyfish`, which implements TurboPlonk over BN254. On top of these libraries, we implement the committed-input checks, a 64-bit secure comparison, Poseidon gadgets compatible with the single-prover circuits, and the settlement session. Each trader generates its own single-prover Groth16 settlement proof using Circom 2.2.3 and `rapidsnark`. The blockchain node is implemented in Go and verifies collaborative proofs through a Rust static library linked using `cgo`.

We ran the experiments on a workstation with 24 logical processors and 29.4 GiB of memory, using Ubuntu 22.04.5 under WSL2. The implementation was compiled in release mode with Rust 1.93.0-nightly and Go 1.26.2. The two traders ran as separate operating-system processes and communicated over QUIC. Unless stated otherwise, both processes and the blockchain node ran on the same machine, and trader traffic used the loopback interface. Proving and verifying keys and the KZG structured reference string were generated before measurement, so setup time is excluded.

For the cryptographic microbenchmark, we perform three warm-up runs followed by 20 measured sessions and report the median and 95th percentile. The network-latency experiment uses three sessions per latency setting, and the end-to-end experiment uses five complete trades.

The current prototype uses mock Beaver triples and a KZG reference string derived from a public seed. The measurements therefore characterize the online execution of the prototype. They exclude secure preprocessing and a production setup ceremony, and the resulting prototype does not provide the cryptographic security guarantees of a production deployment.

B. Cryptographic Protocol Overhead

We first measure the latency and resource use of the collaborative comparison protocol. A single prover evaluating the same relation with the same keys is the baseline. For collaborative execution, we separately measure circuit construction and witness validation (Build), collaborative proving (Prove),

TABLE I
LATENCY OF THE CRYPTOGRAPHIC PHASES IN MILLISECONDS. VALUES ARE MEDIAN OVER 20 MEASURED SESSIONS.

Configuration	Build	Prove	Open	Verify	Total	Total p95
Single prover	n/a	524	n/a	7.4	531	715
2PC, in process	16	1,445	2,869	8.5	4,282	4,617
2PC, QUIC	95	1,317	3,129	6.1	4,537	4,680

TABLE II
RESOURCE CONSUMPTION AND PROOF-SYSTEM SIZES FOR THE COLLABORATIVE COMPARISON.

Metric	Value
Peak memory per trader	1.57 GiB
Peak memory per trader, p95	1.59 GiB
Traffic sent by trader <i>A</i>	60.0 MiB
Traffic sent by trader <i>B</i>	58.2 MiB
TurboPlonk gates	2,048
Public signals	5
Compressed proof size	769 B
Compressed verifying-key size	938 B

authenticated proof opening (Open), and local verification (Verify). We test both an in-process configuration with in-memory channels and two independent processes communicating over QUIC.

Table I shows that the two-process collaborative execution takes 4.54 s, an $8.5\times$ overhead over the 531 ms single-prover baseline. Authenticated opening accounts for 69% of the collaborative latency, while proof generation accounts for 29%; build and verification account for the remaining 2%. The underlying MPC runtime evaluates its dataflow lazily, so opening forces the pending authenticated operations and their communication rounds.

Moving from the in-process configuration to two processes over loopback QUIC increases total latency by 6%. Thus, local transport contributes little in this setting, although this comparison does not predict behavior over a wide-area network. As shown in Table II, each trader uses 1.57 GiB of peak memory, and the two traders send 118.2 MiB in total. The proof is 769 B for the evaluated 2,048-gate relation.

C. Sensitivity to Network Latency

The two-party protocol is interactive, so we next isolate the effect of round-trip latency. A user-space UDP relay delays each datagram by half of the target round-trip time and caps the link at 1 Gbit/s. We evaluate target round-trip times of 0, 10, 30, 60, and 100 ms without changing the protocol or available bandwidth.

As Table III shows, total latency increases from 5.8 s at 0 ms RTT to 33.0 s at 100 ms RTT. The build and opening phases account for this increase because they contain the network-dependent input sharing and authenticated operations. Collaborative proving remains between 1.1 and 1.5 s, and local verification remains below 8 ms across the sweep. One session transfers approximately 118 MiB in both directions, which requires approximately one second at the configured rate. The remaining increase is therefore attributable primarily

TABLE III
CRYPTOGRAPHIC LATENCY UNDER EMULATED NETWORK DELAY IN
MILLISECONDS. VALUES ARE MEDIAN OVER THREE SESSIONS PER
SETTING.

RTT	Build	Prove	Open	Verify	Total	Total p95
0	164	1,420	4,340	7.0	5,794	6,361
10	1,235	1,494	11,253	6.3	13,962	14,083
30	3,285	1,466	15,602	7.6	20,364	22,200
60	6,369	1,363	22,681	7.6	30,446	30,562
100	10,536	1,079	21,325	6.7	33,029	34,460

TABLE IV
END-TO-END LATENCY BY PROTOCOL STEP IN MILLISECONDS. VALUES
ARE MEDIAN OVER FIVE COMPLETE TRADES.

Step	Trader A	Trader B
Order proof	192	193
Order submission confirmation	4,007	6,010
Matching confirmation	4,006	4,006
Preamble fingerprint	2	< 1
Input sharing and binding	43	44
Three-way comparison	14	13
Signature exchange	1	1
Collaborative prove and verify	3,646	3,643
Comparison confirmation	6,010	6,010
Smaller-side reveal	2	2
Payout-note preparation	1	1
Settlement session	10,034	10,034
Settlement submission and confirmation	10,110	10,110

to interaction rounds rather than the bandwidth cap. Because lazy evaluation can move pending work between the build and opening phases, the total latency is the appropriate value for comparing settings.

D. End-to-End Performance and On-Chain Costs

Finally, we execute a complete trade using two trader processes and a proof-verifying blockchain node. We measure order generation and submission, matching, private comparison, comparison confirmation, settlement-proof generation, settlement-message exchange, and final confirmation. The node uses a three-second block interval, and each wallet polls order state every two seconds. The workload contains one maker order selling two units at price three and one taker order buying one unit at the same price. In the implementation’s three-way comparison, the result is +1, and the maker is relisted with the residual quantity.

Tables IV and V show a median settlement latency of 20.1 s and a median full-trade latency of 34.6 s. Collaborative computation and the two order proofs account for 13% of the full-trade latency. Blockchain confirmation and polling account for 58%; the remaining 29% comprises settlement-leg generation, exchange, submission, and confirmation. Consequently, the reported end-to-end result is specific to the configured block and polling intervals.

The node verifies the collaborative proof in 12.37 ms and each single-prover settlement proof in 4.42–4.62 ms. The effective payload for a successful trade is 7,264 B: 1,522 B for each order, 1,999 B for the comparison, and 2,221 B for the combined settlement transaction. Both traders redundantly submit the comparison and settlement messages for liveness; including the duplicate messages rejected by the node, it receives approximately 11,480 B per trade.

TABLE V
END-TO-END AND ON-CHAIN COSTS.

Metric	Median	p95
Settlement latency	20,144 ms	22,094 ms
Full-trade latency	34,553 ms	36,507 ms
Collaborative-proof verification	12.37 ms	13.39 ms
Larger-leg proof verification	4.62 ms	4.70 ms
Smaller-leg proof verification	4.42 ms	4.45 ms
Effective on-chain payload	7,264 B	n/a

E. Limitations

The evaluation has four limitations. First, mock Beaver triples and the development KZG reference string exclude secure preprocessing and a production setup from both the security and performance results. Second, the two traders and blockchain node share one machine; the latency sweep emulates delay but does not reproduce loss, jitter, route changes, or host heterogeneity. Third, the network experiment uses three sessions per point and the end-to-end experiment uses five trades, so their 95th percentiles are descriptive rather than robust tail estimates. The isolated collaborative protocol takes 4.5 s, whereas the same step takes 3.6 s in the end-to-end harness, indicating approximately 20% cross-harness variation. Fourth, the workload covers one comparison circuit and one partially filled trade; it does not measure throughput, concurrent settlements, order-book scaling, or secure preprocessing cost.

VIII. RELATED WORK

Prior work replaces a venue’s trusted operator with secure computation. Cartlidge et al. [7] run the matching engine itself inside MPC, finding a continuous double auction impractical there while volume matching is viable. Prime Match [6] secures matching alone, returning the minimum of two committed quantities; settlement then proceeds through the bank’s ordinary channels.

Invisibook keeps the premise, and as in Prime Match computes on committed inputs proved consistent in zero knowledge, but inverts the placement: matching is public, on posted sides and limit prices, and MPC is confined to settlement—a single comparison between the two counterparties, bound to on-chain commitments. Prime Match’s client-to-client matching is server-aided and the bank learns every matched quantity; Invisibook runs between the counterparties alone and is publicly verifiable on chain.

REFERENCES

- [1] Foundation, Lindsay X. Lin, Legal Counsel at Interstellar and Stellar Development, “Deconstructing Decentralized Exchanges,” *Stanford Journal of Blockchain Law & Policy*, jan 5 2019.
- [2] L. E. Harris, “Order exposure and parasitic traders,” University of Southern California, Working Paper, 1997.
- [3] L. Harris, *Trading and Exchanges: Market Microstructure for Practitioners*. Oxford University Press, 2002.
- [4] dYdX, “dYdX: DeFi’s Pro Trading Platform,” <https://www.dydx.xyz>, 2026, accessed: 2026-08-17.
- [5] Injective, “Injective,” <https://injective.com>, 2026, accessed: 2026-08-17.

- [6] A. Polychroniadou, G. Asharov, B. Diamond, T. Balch, H. Buehler, R. Hua, S. Gu, G. Gimler, and M. Veloso, "Prime match: a privacy-preserving inventory matching system," in *Proceedings of the 32nd USENIX Conference on Security Symposium*, ser. SEC '23, USA, 2023.
- [7] J. Cartledge, N. P. Smart, and Y. Talibi Alaoui, "Mpc joins the dark side," in *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security*, ser. Asia CCS '19, New York, NY, USA, 2019, p. 148–159.
- [8] C. Comerton-Forde and T. J. Putn    , "Dark trading and price discovery," *Journal of Financial Economics*, vol. 118, no. 1, pp. 70–92, 2015.
- [9] H. Bessembinder, M. Panayides, and K. Venkataraman, "Hidden liquidity: An analysis of order exposure strategies in electronic stock markets," *Journal of Financial Economics*, vol. 94, no. 3, pp. 361–383, 2009.
- [10] N. Hautsch and R. Huang, "On the dark side of the market: Identifying and analyzing hidden order placements," Center for Financial Studies, CFS Working Paper 2012/04, 2012.
- [11] H. L. Christensen and R. Woodmansey, "Prediction of hidden liquidity in the limit order book of GLOBEX futures," *The Journal of Trading*, vol. 8, no. 3, 2013.
- [12] D. Zotikov and A. Antonov, "CME iceberg order detection and prediction," *Quantitative Finance*, vol. 21, no. 11, pp. 1977–1992, 2021.
- [13] A. Ozdemir and D. Boneh, "Experimenting with collaborative zk-SNARKs: Zero-Knowledge proofs for distributed secrets," in *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 2022, pp. 4291–4308. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/ozdemir>
- [14] R. Canetti, "Security and composition of multiparty cryptographic protocols," *Journal of Cryptology*, vol. 13, no. 1, pp. 143–202, Jan. 2000.
- [15] N. Atzei, M. Bartoletti, S. Lande, and R. Zunino, "A formal model of bitcoin transactions," in *Financial Cryptography and Data Security*, Berlin, Heidelberg, 2018, pp. 541–560.
- [16] S. Goldwasser and S. Micali, "Probabilistic encryption," *Journal of Computer and System Sciences*, vol. 28, no. 2, pp. 270–299, 1984.
- [17] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof systems," *SIAM Journal on Computing*, vol. 18, no. 1, pp. 186–208, 1989.
- [18] J. Groth, "On the size of pairing-based non-interactive arguments," in *EUROCRYPT 2016, Part II*, ser. LNCS, M. Fischlin and J.-S. Coron, Eds., vol. 9666. Vienna, Austria: Springer Berlin Heidelberg, Germany, May 8–12, 2016, pp. 305–326.
- [19] S. Garg, A. Goel, A. Jain, B. Roberts, and S. Sekar, "Malicious security in collaborative zk-SNARKs: More than meets the eye," in *CRYPTO 2025, Part VII*, ser. LNCS, Y. T. Kalai and S. F. Kamara, Eds., vol. 16006. Santa Barbara, CA, USA: Springer, Cham, Switzerland, Aug. 17–21, 2025, pp. 396–428.
- [20] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *CRYPTO '91*, ser. LNCS, J. Feigenbaum, Ed., vol. 576. Santa Barbara, CA, USA: Springer Berlin Heidelberg, Germany, Aug. 11–15, 1992, pp. 129–140.
- [21] S. Goldwasser, S. Micali, and R. L. Rivest, "A digital signature scheme secure against adaptive chosen-message attacks," *SIAM Journal on Computing*, vol. 17, no. 2, pp. 281–308, Apr. 1988.
- [22] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza, "Zerocash: Decentralized anonymous payments from bitcoin," in *2014 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2014, pp. 459–474.
- [23] D. Hopwood, S. Bowe, T. Hornby, and N. Wilcox, "Zcash protocol specification," Zerocoin Electric Coin Company, Tech. Rep., 2016.
- [24] I. Damg    rd, V. Pastro, N. P. Smart, and S. Zakarias, "Multiparty computation from somewhat homomorphic encryption," in *Advances in Cryptology – CRYPTO 2012*, ser. Lecture Notes in Computer Science, vol. 7417. Springer, 2012, pp. 643–662.